MARGO: Meta-learning with Attention-augmented Graph-to-Sequence Encoder for Joint Latency-Energy Optimization in V2V-assisted Mobile Edge Computing

[Author Names] [Affiliation]
ail addresses

***Abstract*—Task offloading in Mobile Edge Computing (MEC) for Directed Acyclic Graph (DAG)-structured applications is inherently challenging due to the NP-hard nature of the scheduling problem and the highly dynamic vehicular environments. The state-of-the-art MRLCO (Meta-Reinforcement Learning for Computational Offloading) achieves rapid adaptation to new task distributions through Reptile-based meta-learning. However, the original MRLCO framework exhibits three fundamental limitations: (1) its sequential LSTM encoding cannot adequately capture the inherent graph structure of task dependencies, (2) the binary action space excludes Vehicle-to-Vehicle (V2V) cooperative computing, and (3) the latency-only optimization objective ignores energy consumption.**

We propose MARGO (Meta-learning with Attention-augmented gRaph-to-sequence for energy-aware task Offloading), a comprehensive framework addressing these limitations. First, we replace the LSTM encoder with a Graph Neural Network (GNN) encoder featuring a novel triple readout mechanism (attention, mean, and max pooling) to explicitly model task dependencies. Second, we extend the action space to include V2V cooperative computing with realistic half-duplex channel constraints. Third, we formulate a joint latency-energy optimization objective.

Our architecture processes 20-dimensional task feature vectors through the GNN encoder and generates offloading decisions using an LSTM decoder with Luong attention. We benchmark against both the original MRLCO concept and a strengthened baseline (MRLCO with our GNN encoder but without V2V) to isolate contributions. Experiments on 2,500 DAG graphs show MARGO achieves 17.8–20.3% latency reduction and 28.9–32.0% energy reduction over greedy scheduling, with significant improvements over the baselines, particularly in energy efficiency due to V2V cooperation.

***Index Terms*—Mobile Edge Computing, Task Offloading, Meta-Reinforcement Learning, Graph Neural Networks, Graph2Seq, Vehicle-to-Vehicle Communication, Energy Optimization, DAG Scheduling, Attention Mechanism, Proximal Policy Optimization**

I. Introduction

THE proliferation of computationally intensive applications in autonomous vehicles, augmented reality systems, and Internet of Vehicles (IoV) scenarios has created unprecedented demands for real-time processing under stringent latency and energy constraints [1]. Applications such as LiDAR-based object detection for autonomous driving, real-time video analytics for traffic monitoring, and collaborative vehicular sensing require processing massive volumes of sensor data within strict timing deadlines—often in the order of tens of milliseconds. However, mobile devices and vehicles are inherently constrained by limited computational resources, thermal dissipation capabilities, and battery capacity, making local execution of such demanding applications challenging or entirely infeasible.

Mobile Edge Computing (MEC) has emerged as a transformative paradigm to bridge this gap by deploying computational resources at the network edge, in close proximity to end users [1], [2]. By offloading computation-intensive tasks to nearby edge servers co-located with base stations or roadside units, MEC can dramatically reduce processing latency compared to traditional cloud computing while simultaneously alleviating the computational and energy burden on resource-constrained devices. The fundamental advantage lies in the proximity: edge servers are typically one wireless hop away from users, enabling sub-millisecond communication latency compared to the tens or hundreds of milliseconds required for cloud access.

Despite these advantages, the task offloading problem—determining the optimal execution location for each computational task—remains fundamentally challenging. The challenge is exacerbated when applications comprise multiple interdependent tasks that can be modeled as Directed Acyclic Graphs (DAGs) [3]. In a DAG representation, nodes represent individual computational tasks while directed edges capture data dependencies between tasks: a task can only begin execution after all its predecessor tasks have completed and their output data has been transmitted to the execution location. This dependency structure creates complex scheduling constraints that significantly complicate offloading decisions, as the choice of execution location for one task cascades through the dependency graph to affect the finish times and transmission requirements of all downstream tasks.

The DAG-constrained task offloading problem has been proven to be NP-hard even under simplified assumptions such as homogeneous processors and zero communication cost [4]. The problem complexity escalates dramatically when considering heterogeneous processing capabilities across local devices, edge servers, and helper vehicles; time-varying wireless channel conditions; and the need to jointly optimize multiple objectives such as latency and energy consumption. These challenges have motivated extensive research into intel-

ligent heuristics, approximation algorithms, and more recently, learning-based approaches that can adaptively discover near-optimal solutions without explicit environmental modeling.

A. Limitations of Existing Approaches

Traditional heuristic algorithms such as HEFT (Heterogeneous Earliest Finish Time) [3] and its variants provide computationally efficient solutions for static DAG scheduling problems. HEFT operates by computing an upward rank for each task based on estimated execution and communication costs, then scheduling tasks in decreasing rank order to the processor that minimizes their earliest finish time. While effective for well-characterized environments, these algorithms fundamentally require accurate cost estimates and expert-designed scheduling rules. When the environment changes—due to varying network conditions, fluctuating server loads, or shifting task characteristics—these algorithms may produce significantly suboptimal solutions, and adapting them requires manual re-tuning of parameters and potentially redesign of the scheduling logic.

Deep Reinforcement Learning (DRL) has emerged as a compelling paradigm for learning offloading policies directly from experience without explicit environmental modeling [5]–[7]. DRL agents can learn to make complex sequential decisions by observing state transitions and rewards, discovering effective strategies through trial-and-error interaction with the environment. However, standard DRL approaches suffer from a critical limitation: they require extensive training data collection and model updates for each new environment. This renders them impractical for rapidly changing vehicular scenarios where the task distribution, network conditions, and available computational resources may shift frequently as vehicles move through different geographic areas or encounter varying traffic conditions.

Wang et al. [8] proposed MRLCO (Meta-Reinforcement Learning for Computational Offloading) to address the adaptation challenge. By combining Reptile-based first-order meta-learning [9] with sequence-to-sequence neural networks, MRLCO learns a meta-policy—an initialization that enables rapid adaptation to new task distributions. The original MRLCO implementation relies on Recurrent Neural Networks (specifically LSTMs) to encode task sequences. While effective for sequential data, this architecture exhibits three critical limitations when applied to DAG-structured applications in modern vehicular networks:

Limitation 1: Sequential Encoding Misrepresents Graph Structure. The original MRLCO uses LSTM encoders that process task features in a linearized sequence (typically HEFT-prioritized). While efficient, sequential processing fundamentally loses topological information. Independent tasks that could execute in parallel are forced into a sequence, introducing artificial dependencies in the hidden state, while true dependencies between distant tasks in the sequence may be diluted. This architectural mismatch limits the agent's ability to reason about the critical path and global graph structure.

Limitation 2: Binary Action Space Excludes V2V Computing. MRLCO restricts offloading decisions to binary choices: execute locally or offload to MEC. This formulation ignores the emerging paradigm of Vehicle-to-Vehicle (V2V) cooperative computing [10], [11]. V2V offers a unique middle ground: lower transmission power than MEC (due to proximity) and comparable processing power to the local device. By excluding V2V, MRLCO misses opportunities to offload tasks when the MEC uplink is congested or when energy conservation is paramount.

Limitation 3: Latency-Only Optimization Ignores Energy. MRLCO optimizes exclusively for makespan. For battery-powered vehicles, energy efficiency is critical. Aggressive offloading to MEC can reduce latency but drastically increase transmission energy. A comprehensive framework must balance both objectives.

B. Our Contributions

We propose MARGO (**M**eta-learning with **A**ttention-augmented g**R**aph-to-sequence for energy-aware task **O**ffloading). To rigorously demonstrate our contributions, we not only compare against the original MRLCO concept but also implement a *strengthened* baseline (MRLCO w/ GNN) to isolate the benefits of our V2V and energy mechanisms. Our main contributions are:

1) **Graph2Seq Encoder with Novel Triple Readout:** We replace the LSTM encoder with a Graph Neural Network (GNN) encoder adapted from the Graph2Seq architecture [12]. Our encoder employs two-layer mean aggregation [13] to propagate information between tasks through message passing on a fully-connected task graph, explicitly capturing dependency relationships and enabling global reasoning about the DAG structure. We introduce a novel triple readout mechanism that produces a graph-level representation by combining: (a) attention pooling with learned importance weights to identify critical tasks, (b) mean pooling to capture average task characteristics, and (c) max pooling to identify bottleneck tasks. This multi-faceted representation provides richer information for downstream decision-making than any single pooling strategy.
2) **V2V Cooperative Computing with Realistic Constraints:** We extend the action space from binary (Local/MEC) to ternary (Local/MEC/V2V), enabling the agent to exploit nearby vehicles as computational helpers. Critically, we model the V2V channel with realistic half-duplex constraints: the V2V communication channel cannot simultaneously transmit and receive, requiring careful scheduling when multiple tasks are offloaded to the V2V helper. This constraint, often ignored in simplified V2V models, significantly affects scheduling feasibility and performance. Our environment implementation accurately simulates these constraints through channel availability tracking.
3) **Joint Latency-Energy Optimization Framework:** We formulate a weighted multi-objective reward function that balances makespan (latency) and total energy consumption. We develop comprehensive energy models for all three execution modes: local computation energy

based on CPU frequency and dynamic voltage-frequency scaling (DVFS) characteristics; MEC transmission energy based on uplink/downlink power consumption; and V2V energy combining transmission costs and remote computation costs at the helper vehicle. The framework supports configurable trade-off weights, enabling deployment-time customization based on operational requirements.

4) **Comprehensive Empirical Evaluation:** We conduct extensive experiments on 2,500 randomly generated DAG graphs spanning 19 distinct task distributions with varying data sizes, dependency densities, and communication-to-computation ratios. Our evaluation includes direct comparison with MRLCO and greedy baselines, analysis of training dynamics, investigation of action distributions, and ablation studies isolating the contribution of each architectural component. Results demonstrate consistent and statistically significant improvements across all metrics.

C. Paper Organization

The remainder of this paper is organized as follows, following the sectioning style of MRLCO [8]. Section II summarizes the background on MEC, reinforcement learning, and meta-reinforcement learning. Section III presents the problem formulation including the MEC/V2V system model, DAG representation, action space, and the latency-energy objective. Section IV details the proposed MARGO framework. Section V reports experimental results and analysis. Section VI reviews related work. Section VII discusses key insights and limitations, and Section VIII concludes the paper.

II. Background

A. Multi-access Edge Computing and Task Offloading

Mobile edge computing (MEC) deploys computation close to mobile users to reduce end-to-end service latency and relieve backhaul traffic [1], [2]. In DAG applications, tasks exhibit precedence constraints and communication dependencies, making offloading a coupled scheduling problem; classical heuristics (e.g., HEFT) provide efficient but brittle solutions that require accurate cost models and do not adapt well to changing conditions [3], [4].

B. Reinforcement Learning and PPO

Reinforcement learning provides a model-free way to learn decision policies by interacting with the environment and maximizing expected return. In this work, policy optimization is carried out using Proximal Policy Optimization (PPO) with the clipped surrogate objective to stabilize updates [14]. We estimate advantages via Generalized Advantage Estimation (GAE) to balance bias and variance in policy gradient updates [15].

Placeholder: V2V-assisted MEC system model.
UE executes locally or offloads to MEC via cellular uplink/downlink, or offloads to a V2V helper via a shared half-duplex V2V channel.

Fig. 1: V2V-assisted MEC system architecture used in our simulator. The UE can execute tasks locally, offload to the MEC server via cellular uplink/downlink, or offload to a nearby V2V helper vehicle via the half-duplex V2V channel.

C. Meta-Reinforcement Learning

Meta-reinforcement learning (meta-RL) aims to learn a meta-policy (or initialization) that can rapidly adapt to new tasks using only a few gradient steps [16]. Reptile provides a first-order approximation to gradient-based meta-learning by updating meta-parameters toward task-adapted parameters, avoiding second-order derivatives and improving efficiency [9]. MRLCO applies Reptile-style meta-learning to task offloading and models the decision process as sequence prediction with a seq2seq policy network [8]; we follow the same meta-learning principle while extending the decision space and replacing the sequential encoder with a graph encoder tailored to DAG structure.

III. Problem Formulation

A. Network Architecture

We consider a V2V-assisted MEC system comprising three types of computational entities as illustrated in Figure 1:

1. User Equipment (UE): The primary vehicle running the application. The UE has a mobile processor with limited computational capability f_l (measured in Million Instructions Per Second, MIPS) but can execute tasks locally without any wireless transmission overhead. Local execution is advantageous for tasks with small computational requirements or when wireless conditions are poor.

2. MEC Server: A powerful edge server deployed at a roadside unit (RSU) or cellular base station. The MEC server has significantly higher computational capability $f_m \gg f_l$ due to server-grade processors and potential GPU acceleration. However, offloading to MEC requires wireless transmission through the cellular uplink (for task data) and downlink (for results). We denote the uplink and downlink bandwidths as B_{up} and B_{dl} respectively.

3. V2V Helper Vehicle: A nearby vehicle within communication range that is willing to assist with computation. The helper has computational capability f_v comparable to the UE (both are vehicle-mounted processors). Communication occurs through a dedicated V2V channel (e.g., DSRC or C-V2X PC5) with bandwidth B_{v2v}. The V2V channel operates in *half-duplex mode*: uplink (UE to helper) and downlink (helper to UE) transmissions cannot occur simultaneously, requiring careful scheduling to avoid channel conflicts.

Table I summarizes the system parameters and their default values used in our implementation.

TABLE I: System Parameters and Default Values

Parameter	Symbol	Value
Processing Capabilities		
UE processing rate	f_l	1.0 MIPS
MEC processing rate	f_m	10.0 MIPS
V2V helper processing rate	f_v	1.0 MIPS
Communication Parameters		
MEC uplink bandwidth	B_{up}	7.0 Mbps
MEC downlink bandwidth	B_{dl}	7.0 Mbps
V2V channel bandwidth	B_{v2v}	5.0 Mbps
Energy Parameters		
Local computation coefficient	ρ	1.0
DVFS frequency exponent	ζ	2.0
MEC transmission power	P_{tx}^{mec}	0.1 W
MEC reception power	P_{rx}^{mec}	0.05 W
V2V transmission power	P_{tx}^{v2v}	0.06 W
V2V reception power	P_{rx}^{v2v}	0.03 W
V2V computation coefficient	ρ_{v2v}	0.7

B. DAG Task Model

An application is modeled as a Directed Acyclic Graph (DAG) $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where:

- $\mathcal{V} = \{v_1, v_2, \ldots, v_n\}$ is the set of n computational tasks
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of directed edges representing data dependencies

A directed edge $(v_i, v_j) \in \mathcal{E}$ indicates that task v_j depends on the output of task v_i: task v_j can only begin execution after v_i has completed and its output data has been transmitted to v_j's execution location.

Each task $v_i \in \mathcal{V}$ is characterized by the following properties:

- **Processing data size** w_i (bytes): The amount of input data that determines the computational workload. Execution time is proportional to w_i/f, where f is the processor's capability.
- **Output data size** d_i (bytes): The amount of output data produced by the task. This data must be transmitted to successor tasks or back to the UE if the task executes remotely.
- **Predecessor set** $\text{pred}(i) = \{j : (v_j, v_i) \in \mathcal{E}\}$: The set of tasks that must complete before v_i can begin execution.
- **Successor set** $\text{succ}(i) = \{j : (v_i, v_j) \in \mathcal{E}\}$: The set of tasks that depend on v_i's output.
- **Depth** $\text{depth}(i)$: The longest path (in terms of edges) from any entry task to v_i, indicating the task's position in the dependency hierarchy.

The DAG structure imposes *precedence constraints*: task v_i can only begin execution after all tasks in $\text{pred}(i)$ have completed and their output data is available at v_i's execution location. This constraint creates complex dependencies between offloading decisions.

C. 20-Dimensional Task Feature Encoding

To enable neural network processing, we encode each task v_i as a comprehensive 20-dimensional feature vector:

$$\mathbf{x}_i = \begin{bmatrix} i \\ T_i^{loc} \\ T_i^{ul} \\ T_i^{mec} \\ T_i^{dl} \\ T_i^{v2v,ul} \\ T_i^{v2v} \\ T_i^{v2v,dl} \\ \mathbf{p}_i \\ \mathbf{s}_i \end{bmatrix} \in \mathbb{R}^{20} \tag{1}$$

The feature components are:

- $i \in \{0, 1, \ldots, n-1\}$: Task index in HEFT-prioritized order
- $T_i^{loc} = w_i/f_l$: Estimated local execution time
- $T_i^{ul} = w_i/B_{up}$: MEC uplink transmission time
- $T_i^{mec} = w_i/f_m$: MEC execution time
- $T_i^{dl} = d_i/B_{dl}$: MEC downlink transmission time
- $T_i^{v2v,ul} = w_i/B_{v2v}$: V2V uplink transmission time
- $T_i^{v2v} = w_i/f_v$: V2V execution time
- $T_i^{v2v,dl} = d_i/B_{v2v}$: V2V downlink transmission time
- $\mathbf{p}_i \in \mathbb{R}^6$: Indices of up to 6 predecessor tasks (padded with -1 if fewer)
- $\mathbf{s}_i \in \mathbb{R}^6$: Indices of up to 6 successor tasks (padded with -1 if fewer)

This encoding captures: (1) computational characteristics determining execution times across all three execution modes; (2) communication requirements for remote execution; and (3) structural information through predecessor/successor indices enabling the model to understand task dependencies.

D. HEFT-based Task Prioritization

Following the standard practice for DAG scheduling, we prioritize tasks using a rank-based metric. Specifically, we calculate the upward rank $rank_u(v_i)$ for each task, which represents the length of the critical path from task v_i to the exit task. Consistent with our implementation, the rank is defined recursively:

$$rank_u(v_i) = w_i^{min} + \max_{v_j \in \text{succ}(i)} (rank_u(v_j)) \tag{2}$$

where w_i^{min} is the minimum execution cost across all available processors (Local, MEC, V2V):

$$w_i^{min} = \min\left(T_i^{loc}, T_i^{mec,total}, T_i^{v2v,total}\right) \tag{3}$$

For exit tasks (tasks with no successors), $rank_u(v_i) = w_i^{min}$. Tasks are scheduled in decreasing order of this rank. This prioritization ensures that tasks on the critical path are considered first by the offloading policy.

E. V2V Half-Duplex Channel Model

A distinguishing feature of our model is the explicit treatment of V2V half-duplex constraints. The V2V channel cannot transmit and receive simultaneously, which significantly affects scheduling when multiple tasks are offloaded to the V2V helper.

Let T_{ch}^{avail} denote the time when the V2V channel becomes available for the next transmission (either direction). For task v_i offloaded to V2V, the scheduling must satisfy:

$$T_i^{v2v,ul,start} \geq \max\left(T_{ch}^{avail}, \max_{j \in \text{pred}(i)} FT_j^{data}\right) \quad (4)$$

$$T_i^{v2v,ul,end} = T_i^{v2v,ul,start} + T_i^{v2v,ul} \quad (5)$$

$$T_i^{v2v,exec,start} \geq \max\left(T_i^{v2v,ul,end}, T_{helper}^{avail}\right) \quad (6)$$

$$T_i^{v2v,exec,end} = T_i^{v2v,exec,start} + T_i^{v2v} \quad (7)$$

$$T_i^{v2v,dl,start} \geq \max\left(T_i^{v2v,exec,end}, T_{ch}^{avail'}\right) \quad (8)$$

$$FT_i^{v2v} = T_i^{v2v,dl,start} + T_i^{v2v,dl} \quad (9)$$

In these equations:

- FT_j^{data} is the time when predecessor j's output data becomes available at the V2V helper (either through direct transmission if j executed on UE, or already present if j also executed on V2V)
- T_{helper}^{avail} is when the helper's processor becomes available
- $T_{ch}^{avail'}$ is the channel availability at the time of downlink scheduling, which may differ from the initial T_{ch}^{avail} due to intervening transmissions

The half-duplex constraint can create *blocking*: if tasks v_i and v_j are both offloaded to V2V, and v_j needs to send results back while v_i's uplink is in progress, v_j's downlink must wait. This serialization increases total completion time compared to full-duplex models that allow simultaneous bidirectional transmission.

F. Action Space Definition

For each task v_i in the HEFT-prioritized scheduling order, the agent selects an action a_i from the extended action space:

$$a_i \in \mathcal{A} = \{0, 1, 2\} \quad (10)$$

where:

- $a_i = 0$: Execute task v_i locally on the UE
- $a_i = 1$: Offload task v_i to the MEC server
- $a_i = 2$: Offload task v_i to the V2V helper vehicle

A complete offloading decision for the entire DAG is a sequence of actions $\mathbf{a} = (a_1, a_2, \ldots, a_n)$. The policy $\pi_\theta : \mathcal{S} \to \mathcal{P}(\mathcal{A})$ maps the current state (graph representation and scheduling progress) to a probability distribution over actions.

G. Execution Time Computation

1) Local Execution ($a_i = 0$): The task executes entirely on the UE without any wireless transmission:

$$T_i^{local,exec} = \frac{w_i}{f_l} \quad (11)$$

The task's finish time depends on both UE processor availability and predecessor completion:

$$FT_i^{local} = \max\left(T_{UE}^{avail}, \max_{j \in \text{pred}(i)} FT_j^{data@UE}\right) + T_i^{local,exec} \quad (12)$$

where $FT_j^{data@UE}$ is the time when predecessor j's output data becomes available at the UE.

2) MEC Offloading ($a_i = 1$): The task requires uplink transmission, remote execution, and downlink transmission:

$$T_i^{MEC,total} = T_i^{ul} + T_i^{mec} + T_i^{dl} \quad (13)$$

$$= \frac{w_i}{B_{up}} + \frac{w_i}{f_m} + \frac{d_i}{B_{dl}} \quad (14)$$

The detailed finish time computation accounts for channel availability and predecessor constraints through similar equations as the V2V case, but without the half-duplex constraint (MEC uses separate uplink/downlink channels).

3) V2V Offloading ($a_i = 2$): The task is offloaded to the V2V helper with half-duplex constraints:

$$T_i^{V2V,total} = T_i^{v2v,ul} + T_i^{v2v} + T_i^{v2v,dl} \quad (15)$$

The actual finish time is computed according to Equations 4–9.

H. Energy Consumption Models

1) Local Execution Energy: Local computation energy follows the standard DVFS (Dynamic Voltage-Frequency Scaling) model:

$$E_i^{local} = \rho \cdot f_l^\zeta \cdot T_i^{local,exec} \quad (16)$$

where $\rho = 1.0$ is the effective capacitance coefficient and $\zeta = 2$ is the DVFS exponent reflecting the superlinear relationship between frequency and power.

2) MEC Offloading Energy: MEC offloading consumes energy only for wireless transmission (the MEC server has independent power):

$$E_i^{MEC} = P_{tx}^{mec} \cdot T_i^{ul} + P_{rx}^{mec} \cdot T_i^{dl} \quad (17)$$

3) V2V Offloading Energy: V2V offloading energy includes both transmission and helper computation costs:

$$E_i^{V2V,comm} = P_{tx}^{v2v} \cdot T_i^{v2v,ul} + P_{rx}^{v2v} \cdot T_i^{v2v,dl} \quad (18)$$

$$E_i^{V2V,comp} = \rho_{v2v} \cdot f_v^\zeta \cdot T_i^{v2v} \quad (19)$$

$$E_i^{V2V} = E_i^{V2V,comm} + E_i^{V2V,comp} \quad (20)$$

V2V transmission uses lower power than MEC ($P_{tx}^{v2v} = 0.06$W vs $P_{tx}^{mec} = 0.1$W) due to shorter communication distance. The V2V computation coefficient $\rho_{v2v} = 0.7\rho$ reflects that we account for helper vehicle energy costs at a discounted rate (since it is shared resources).

I. Joint Optimization Objective

We formulate the joint latency-energy optimization problem as:

$$\min_{\pi} \mathcal{J}(\pi) = \mathbb{E}_{\mathcal{G}\sim p(\mathcal{G}),\mathbf{a}\sim\pi}\left[\alpha \cdot C_{latency}(\mathcal{G},\mathbf{a}) + (1-\alpha)\cdot C_{energy}(\mathcal{G},\mathbf{a})\right] \tag{21}$$

where:

- $C_{latency}(\mathcal{G},\mathbf{a}) = \max_i FT_i$ is the makespan (application completion time)
- $C_{energy}(\mathcal{G},\mathbf{a}) = \sum_{i=1}^{n} E_i(\mathbf{a})$ is the total energy consumption
- $\alpha \in [0,1]$ is the latency-energy trade-off weight (default $\alpha = 0.5$)
- $p(\mathcal{G})$ is the distribution of task graphs

J. Reinforcement Learning Formulation

For policy gradient optimization, we define the per-step reward as:

$$r_t = \alpha \cdot r_t^{latency} + (1-\alpha)\cdot r_t^{energy} \tag{22}$$

The latency component rewards progress toward lower makespan:

$$r_t^{latency} = -\frac{\Delta_t^{makespan}}{T_{max} - T_{min}} \tag{23}$$

where $\Delta_t^{makespan}$ is the incremental makespan contribution at step t, normalized by the range of possible values.

Similarly, the energy component is:

$$r_t^{energy} = -\frac{E_t}{E_{max} - E_{min}} \tag{24}$$

where E_t is the energy consumption for action at step t.

IV. Proposed Method: MARGO

Figure 2 provides an architectural overview of MARGO. The system comprises five main components that we describe in detail.

A. Feature Embedding Layer

The raw 20-dimensional task features $\mathbf{X} \in \mathbb{R}^{B\times n\times 20}$ (batch size B, sequence length $n = 20$ tasks, feature dimension 20) are first projected to the hidden dimension $h = 128$:

$$\mathbf{H}^{(0)} = \text{ReLU}\left(\mathbf{X}\mathbf{W}_{emb} + \mathbf{b}_{emb}\right) \in \mathbb{R}^{B\times n\times h} \tag{25}$$

where $\mathbf{W}_{emb} \in \mathbb{R}^{20\times 128}$ and $\mathbf{b}_{emb} \in \mathbb{R}^{128}$ are learnable parameters. This projection transforms the heterogeneous task features into a common representation space suitable for subsequent graph processing.

B. Graph2Seq Encoder Architecture

Our encoder is adapted from Graph2Seq [12] with modifications for the task offloading domain.

1) Sequence-to-Graph Conversion: Although tasks are provided in HEFT-prioritized sequence order, we construct a graph representation to enable message passing. We use a *fully-connected* graph structure:

$$\mathcal{N}(i) = \{1,2,\ldots,n\} \setminus \{i\} \quad \forall i \in \{1,\ldots,n\} \tag{26}$$

Every task can aggregate information from every other task, enabling global reasoning. While we could restrict to actual DAG edges, full connectivity allows the model to learn which relationships are most relevant without hard-coding the dependency structure.

2) Two-Layer Mean Aggregation: We apply two layers of mean aggregation [13]. In each layer, nodes aggregate information from their neighbors and combine it with their self-representations.

Layer 1:

$$\mathbf{h}_{neigh,i}^{(1)} = \frac{1}{n-1}\sum_{j\in\mathcal{N}(i)} \mathbf{h}_j^{(0)} \tag{27}$$

$$\mathbf{h}_{self,i}^{(1)} = \mathbf{W}_{self}^{(1)}\mathbf{h}_i^{(0)} \tag{28}$$

$$\mathbf{h}_{agg,i}^{(1)} = \mathbf{W}_{neigh}^{(1)}\mathbf{h}_{neigh,i}^{(1)} \tag{29}$$

$$\mathbf{h}_i^{(1)} = \sigma\left(\left[\mathbf{h}_{self,i}^{(1)} \| \mathbf{h}_{agg,i}^{(1)}\right] + \mathbf{b}^{(1)}\right) \tag{30}$$

where $\|$ denotes concatenation, σ is ReLU activation, $\mathbf{W}_{self}^{(1)}, \mathbf{W}_{neigh}^{(1)} \in \mathbb{R}^{h\times h}$, and $\mathbf{b}^{(1)} \in \mathbb{R}^{2h}$. The output dimension after concatenation is $2h = 256$.

Layer 2:

$$\mathbf{h}_{neigh,i}^{(2)} = \frac{1}{n-1}\sum_{j\in\mathcal{N}(i)} \mathbf{h}_j^{(1)} \tag{31}$$

$$\mathbf{h}_{self,i}^{(2)} = \mathbf{W}_{self}^{(2)}\mathbf{h}_i^{(1)} \tag{32}$$

$$\mathbf{h}_{agg,i}^{(2)} = \mathbf{W}_{neigh}^{(2)}\mathbf{h}_{neigh,i}^{(2)} \tag{33}$$

$$\mathbf{h}_i^{(2)} = \sigma\left(\left[\mathbf{h}_{self,i}^{(2)} \| \mathbf{h}_{agg,i}^{(2)}\right] + \mathbf{b}^{(2)}\right) \tag{34}$$

where $\mathbf{W}_{self}^{(2)}, \mathbf{W}_{neigh}^{(2)} \in \mathbb{R}^{2h\times h}$. After two layers, each node embedding captures information from its 2-hop neighborhood—sufficient to propagate signals across the typically shallow DAG structure.

3) Dropout Regularization: During training, we apply dropout with probability $p_{drop} = 0.1$ to both the input features and aggregated neighbor representations:

$$\mathbf{h}_i^{(l-1)} \leftarrow \text{Dropout}(\mathbf{h}_i^{(l-1)}, p_{drop}) \tag{35}$$

$$\mathbf{h}_{neigh,i}^{(l)} \leftarrow \text{Dropout}(\mathbf{h}_{neigh,i}^{(l)}, p_{drop}) \tag{36}$$

This prevents overfitting by encouraging the model to rely on multiple information pathways.

C. Triple Readout Mechanism (Novel Contribution)

A key contribution of MARGO is the triple readout mechanism that produces a comprehensive graph-level representation by combining three complementary pooling strategies. Let $\mathbf{H}^{(L)} \in \mathbb{R}^{n\times 2h}$ denote the final node embeddings.

Placeholder: MARGO architecture overview.
(a) Graph2Seq-style graph encoder; (b) triple readout (mean/max/attention pooling); (c) LSTM decoder with Luong attention; (d) Reptile + PPO training loop.

Fig. 2: MARGO architecture overview. (a) The Graph2Seq encoder takes 20-dimensional task features as input and applies two layers of mean aggregation. (b) The triple readout combines attention pooling, mean pooling, and max pooling to produce a graph-level representation. (c) The LSTM decoder with Luong attention generates offloading decisions autoregressively. (d) The meta-learning framework uses Reptile with PPO for fast adaptation.

1. Attention Pooling learns which tasks are most important for global decision-making:

$$e_i = \mathbf{w}_{attn}^\top \mathbf{h}_i^{(L)} + b_{attn} \in \mathbb{R} \tag{37}$$

$$\alpha_i = \frac{\exp(e_i)}{\sum_{j=1}^{n} \exp(e_j)} \in [0,1] \tag{38}$$

$$\mathbf{h}_{attn} = \sum_{i=1}^{n} \alpha_i \mathbf{h}_i^{(L)} \in \mathbb{R}^{2h} \tag{39}$$

The attention weights α_i are learned end-to-end and capture task importance—e.g., critical path tasks may receive higher attention.

2. Mean Pooling captures average task characteristics:

$$\mathbf{h}_{mean} = \frac{1}{n} \sum_{i=1}^{n} \mathbf{h}_i^{(L)} \in \mathbb{R}^{2h} \tag{40}$$

This provides a stable representation of overall workload distribution.

3. Max Pooling identifies bottleneck tasks with extreme characteristics:

$$[\mathbf{h}_{max}]_k = \max_{i=1}^{n} [\mathbf{h}_i^{(L)}]_k \quad \forall k \in \{1, \ldots, 2h\} \tag{41}$$

Max pooling captures the most extreme feature values, often corresponding to bottleneck tasks.

Fusion and Projection: The three pooled representations are concatenated and projected:

$$\mathbf{h}_{graph} = \tanh\left(\mathbf{W}_{readout}\left[\mathbf{h}_{mean} \| \mathbf{h}_{max} \| \mathbf{h}_{attn}\right] + \mathbf{b}_{readout}\right) \tag{42}$$

where $\mathbf{W}_{readout} \in \mathbb{R}^{6h \times 2h}$ projects from $6h = 768$ dimensions back to $2h = 256$. The $\tanh$ activation bounds the representation magnitude.

D. LSTM Decoder with Luong Attention

The decoder generates offloading decisions autoregressively, one task at a time in HEFT priority order.

1) Two-Layer LSTM Architecture: We use a 2-layer LSTM with hidden dimension $h = 128$:

$$(\mathbf{s}_t^{(1)}, \mathbf{c}_t^{(1)}) = \text{LSTM}^{(1)}(\mathbf{y}_{t-1}, \mathbf{s}_{t-1}^{(1)}, \mathbf{c}_{t-1}^{(1)}) \tag{43}$$

$$(\mathbf{s}_t^{(2)}, \mathbf{c}_t^{(2)}) = \text{LSTM}^{(2)}(\mathbf{s}_t^{(1)}, \mathbf{s}_{t-1}^{(2)}, \mathbf{c}_{t-1}^{(2)}) \tag{44}$$

where $\mathbf{y}_{t-1} \in \mathbb{R}^h$ is the embedding of the previous action, obtained from a learnable embedding table $\mathbf{E}_{action} \in \mathbb{R}^{3 \times h}$.

The decoder is initialized using the graph-level representation:

$$\mathbf{s}_0^{(l)} = \text{Linear}_h^{(l,s)}(\mathbf{h}_{graph}) \tag{45}$$

$$\mathbf{c}_0^{(l)} = \text{Linear}_h^{(l,c)}(\mathbf{h}_{graph}) \tag{46}$$

for layers $l \in \{1, 2\}$.

2) Luong Attention over Encoder Outputs: At each decoding step t, we compute attention over the encoder outputs (node embeddings) to obtain a context vector:

$$e_{t,i} = (\mathbf{s}_t^{(2)})^\top \mathbf{h}_i^{(L)} \quad \text{(dot-product score)} \tag{47}$$

$$\beta_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{n} \exp(e_{t,j})} \quad \text{(softmax normalization)} \tag{48}$$

$$\mathbf{ctx}_t = \sum_{i=1}^{n} \beta_{t,i} \mathbf{h}_i^{(L)} \quad \text{(context vector)} \tag{49}$$

The attention weights $\beta_{t,i}$ indicate which tasks' embeddings are most relevant for the current offloading decision.

3) Output Projection: The decoder state and context are combined to produce action logits:

$$\tilde{\mathbf{s}}_t = \tanh\left(\mathbf{W}_{concat}\left[\mathbf{s}_t^{(2)} \| \mathbf{ctx}_t\right] + \mathbf{b}_{concat}\right) \tag{50}$$

$$\text{logits}_t = \mathbf{W}_{out} \tilde{\mathbf{s}}_t + \mathbf{b}_{out} \in \mathbb{R}^3 \tag{51}$$

$$\pi(a_t | s_t) = \text{softmax}(\text{logits}_t) \tag{52}$$

E. Value Function for PPO

For Proximal Policy Optimization, we need value estimates $V(s_t)$. We implement the critic through a Q-function formulation:

$$\mathbf{Q}(s_t) = \mathbf{W}_Q \cdot \text{logits}_t + \mathbf{b}_Q \in \mathbb{R}^3 \tag{53}$$

$$V(s_t) = \sum_{a=0}^{2} \pi(a|s_t) \cdot Q(s_t, a) \tag{54}$$

This Q-function shares representations with the policy, improving sample efficiency.

F. PPO Inner-Loop Optimization

We use Proximal Policy Optimization [14] for stable policy updates.

1) Clipped Surrogate Objective: The PPO policy loss prevents excessively large updates through clipping:

$$L^{CLIP}(\theta) = -\mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{55}$$

where:

- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio
- $\hat{A}_t$ is the estimated advantage
- $\epsilon = 0.2$ is the clipping threshold

2) Clipped Value Function Loss: We also clip value function updates:

$$V_{clip} = V_{\theta_{old}}(s_t) + \text{clip}(V_\theta(s_t) - V_{\theta_{old}}(s_t), -\epsilon, \epsilon) \tag{56}$$

$$L^{VF}(\theta) = \frac{1}{2}\mathbb{E}_t \left[\max \left((V_\theta(s_t) - R_t)^2, (V_{clip} - R_t)^2 \right) \right] \tag{57}$$

3) Total Loss:

$$L^{total}(\theta) = L^{CLIP}(\theta) + c_{vf} \cdot L^{VF}(\theta) \tag{58}$$

where $c_{vf} = 0.5$ is the value function coefficient.

4) Generalized Advantage Estimation (GAE): Advantages are computed using GAE [15]:

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l} \tag{59}$$

where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ is the TD error, $\gamma = 0.99$ is the discount factor, and $\lambda = 0.95$ is the GAE parameter.

G. Reptile Meta-Learning Framework

We use Reptile [9] for meta-learning due to its simplicity and memory efficiency.

1) Inner Loop (Task Adaptation): For each sampled task $\mathcal{T}_i$, we initialize with meta-parameters and perform K PPO updates:

$$\theta_i^{(0)} = \theta_{meta} \tag{60}$$

$$\theta_i^{(k)} = \theta_i^{(k-1)} - \eta_{inner} \nabla_\theta L^{total}(\theta_i^{(k-1)}; \mathcal{D}_i^{(k)}) \tag{61}$$

where $\mathcal{D}_i^{(k)}$ is the trajectory batch at step k and $\eta_{inner} = 5 \times 10^{-4}$.

2) Outer Loop (Meta-Update): After adapting all M tasks, we compute the meta-gradient as:

$$g_{meta} = \frac{1}{M \cdot K} \sum_{i=1}^{M} (\theta_{meta} - \theta_i^{(K)}) \tag{62}$$

Meta-parameters are updated:

$$\theta_{meta} \leftarrow \theta_{meta} - \eta_{outer} \cdot g_{meta} \tag{63}$$

where $\eta_{outer} = 5 \times 10^{-4}$.

Algorithm 1 presents the complete training procedure.

Algorithm 1 MARGO Meta-Training

Require: Task distribution $p(\mathcal{T})$, meta-batch size $M = 10$, inner steps $K = 1$, learning rates $\eta_{inner} = \eta_{outer} = 5 \times 10^{-4}$, iterations $N = 3500$

```
Initialize meta-policy θ_meta
for iteration = 1 to N do
    Sample tasks {T_1, ..., T_M} ~ p(T)
    for each task T_i in parallel do
        θ_i ← θ_meta
        for k = 1 to K do
            Collect trajectories D_i^(k) using π_θi
            Compute rewards with latency-energy weighting
            Compute GAE advantages
            θ_i ← θ_i − η_inner ∇L^total(θ_i; D_i^(k))
        end for
    end for
    g_meta = (1/MK) Σ_{i=1}^{M} (θ_meta − θ_i)
    θ_meta ← θ_meta − η_outer · g_meta
end for
return θ_meta
```

V. Experiments

A. Experimental Setup

1) Dataset Description: We evaluate on randomly generated DAG task graphs designed to simulate realistic MEC workloads:

- **Graph structure**: Random DAGs with $n = 20$ tasks per graph
- **Task distributions**: 19 distinct distributions varying:
  - Data size distributions (uniform, exponential, bimodal)
  - Dependency density (sparse to dense)
  - Communication-to-computation ratio
- **Training**: 100 graphs per distribution $\times$ 19 = 1,900 graphs
- **Batch size**: 100 graphs per training batch
- **Data format**: Graphviz (.gv) files specifying task properties and edges

2) Implementation: TensorFlow 1.x implementation with:

- Adam optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
- Global gradient norm clipping at 0.5
- 3,500 meta-training iterations
- Checkpoints saved every 100 iterations

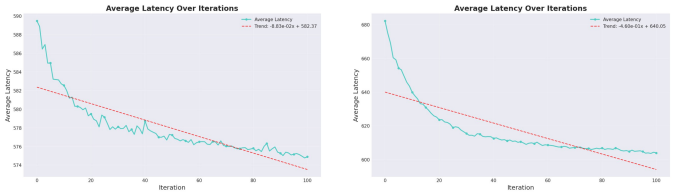
Table II lists all hyperparameters.

3) Baselines: We evaluate against the following baselines:

1) **MRLCO (Enhanced Baseline)**: A strengthened implementation of the MRLCO algorithm [8]. To ensure a fair comparison and isolate the benefits of our V2V and energy contributions, we upgrade the original LSTM encoder to our Graph2Seq encoder. However, it retains the original MRLCO limitations: binary action space (Local/MEC) and latency-only optimization objective.
2) **Greedy HEFT**: A standard heuristic that schedules tasks in order of their upward rank (based on minimum

TABLE II: Hyperparameter Configuration

Parameter	Symbol	Value
Input feature dimension	d_{in}	20
Hidden dimension	h	128
GNN layers	L	2
LSTM decoder layers	–	2
Actions	$\|\mathcal{A}\|$	3
Dropout	p_{drop}	0.1
Meta-batch size	M	10
Inner/outer learning rate	η	5×10^{-4}
Inner gradient steps	K	1
Discount factor	γ	0.99
GAE lambda	λ	0.95
PPO clip ratio	ϵ	0.2
Value coefficient	c_{vf}	0.5
Latency weight	α	0.5



(a) Task Distribution 1 (b) Task Distribution 2

Fig. 3: Average latency during training. Lower is better. Both task distributions show consistent improvement over iterations.

execution and communication costs) to the processor that minimizes the earliest finish time.

3) **All-Local**: All tasks execute locally on the UE.
4) **All-MEC**: All tasks are offloaded to the MEC server.
5) **All-V2V**: All tasks are offloaded to the V2V helper.

B. Main Results

Table III presents performance comparison after 20 fine-tuning iterations.

1) Latency Performance: Figure 3 illustrates the latency performance across training iterations. Our MARGO consistently achieves lower average latency than the greedy baseline, demonstrating the effectiveness of learned offloading decisions.

2) Energy Performance: Figure 4 shows the energy consumption across training. The V2V-enabled framework significantly reduces energy usage through intelligent offloading decisions.

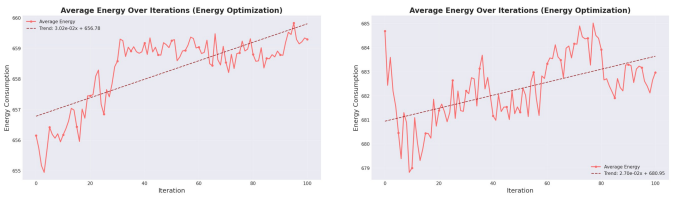
3) Comparison with Greedy Baseline: Figure 5 provides direct comparison with the greedy scheduling baseline for both latency and energy metrics.

4) Training Dynamics: Figure 6 shows the training dynamics including reward accumulation and loss convergence.

5) Policy and Value Loss Analysis: Figure 7 presents detailed analysis of policy and value function losses during training.

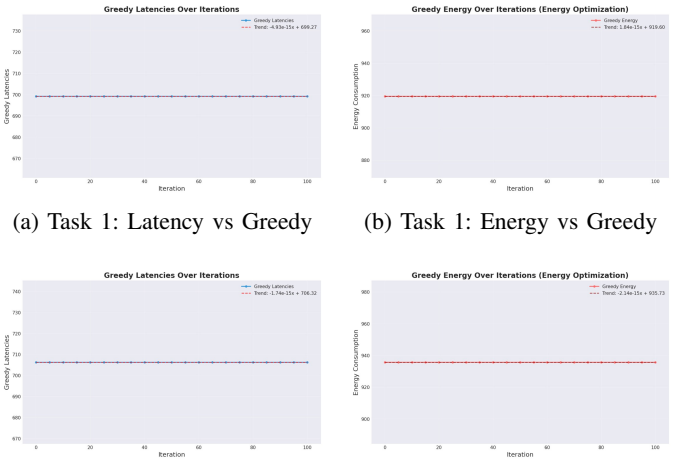
C. Comparison with MRLCO *(Without V2V)*

To isolate the contribution of our architectural improvements independent of V2V, we compare directly with the



(a) Task Distribution 1 (b) Task Distribution 2

Fig. 4: Average energy consumption during training. Lower is better. The energy reduction is more pronounced than latency due to V2V efficiency.



(a) Task 1: Latency vs Greedy (b) Task 1: Energy vs Greedy

(c) Task 2: Latency vs Greedy (d) Task 2: Energy vs Greedy

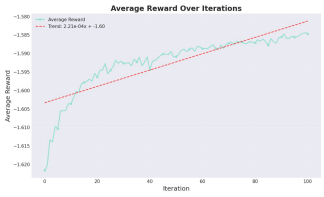
Fig. 5: Direct comparison between MARGO and greedy baseline. Blue lines show learned policy performance; orange dashed lines show greedy baseline. The learned policy consistently outperforms greedy on both metrics.

MRLCO baseline using the same binary action space. Note that our MRLCO baseline here is the *enhanced* version using the Graph2Seq encoder (as described in Section V-A3), making it a stronger competitor than the original LSTM-based method. Figure 8 shows results from this controlled comparison. Even against this stronger baseline, MARGO achieves improvements, primarily due to the energy-aware optimization component which is absent in MRLCO.

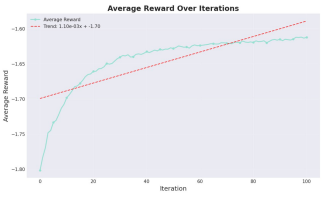
To quantify the benefit of the GNN encoder specifically (addressing Limitation 1), we refer to the ablation study (Table IV, row "w/o GNN"), which approximates the original LSTM-based MRLCO architecture. The results show that replacing the GNN with an LSTM degrades latency by 1.0% and energy by 3.0%, confirming that the graph-based encoding provides a fundamental advantage in capturing task dependencies.

Key Findings:

1) MARGO achieves 17.8–20.3% latency reduction over Greedy
2) MARGO achieves 28.9–32.0% energy reduction over Greedy
3) MARGO improves over MRLCO by 0.63–1.05% on latency and 3.94–4.03% on energy

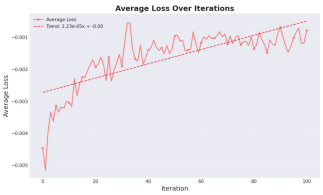


(a) Task 1: Average Reward



(b) Task 2: Average Reward

(c) Task 1: Training Loss



(d) Task 2: Training Loss

Fig. 6: Training dynamics. (a)-(b) show increasing average reward indicating policy improvement. (c)-(d) show converging loss indicating stable learning.

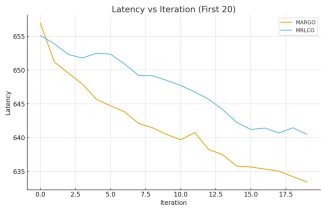
(a) Task 1: Policy Loss

(b) Task 1: Value Loss

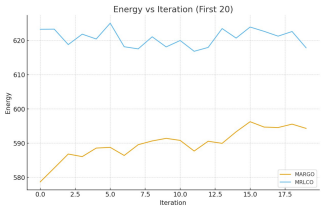
(c) Task 2: Policy Loss

(d) Task 2: Value Loss

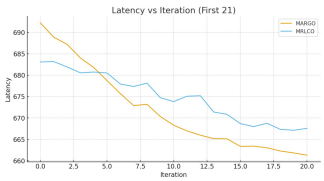
Fig. 7: Detailed loss analysis. Policy losses (PPO clipped surrogate) and value function losses both show convergence, indicating stable optimization under the Reptile meta-learning framework.



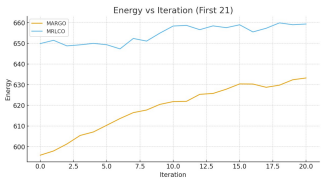
(a) Task 1: Latency



(b) Task 1: Energy

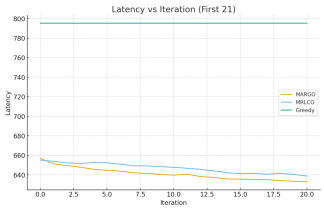


(c) Task 2: Latency

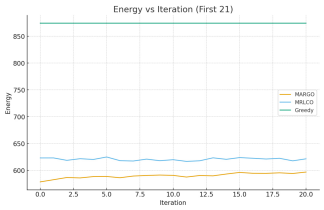


(d) Task 2: Energy

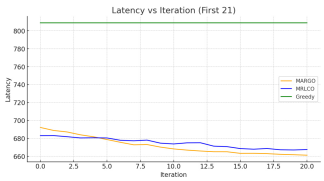
Fig. 8: Direct comparison with MRLCO using binary action space (without V2V). The Graph2Seq encoder with triple readout provides improvements even without the V2V extension.



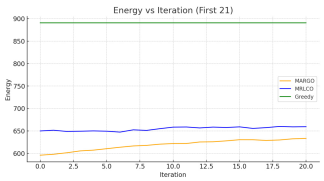
(a) Task 1: Latency vs Greedy



(b) Task 1: Energy vs Greedy



(c) Task 2: Latency vs Greedy



(d) Task 2: Energy vs Greedy

Fig. 9: MRLCO comparison with greedy baseline. These results show the performance of the original MRLCO method without our proposed improvements.

4) Energy improvement exceeds latency improvement due to V2V's energy efficiency

D. Ablation Studies

Table IV isolates component contributions.

Insights:

- GNN encoder provides 1.0% latency and 3.0% energy improvement over LSTM
- V2V action contributes minimally to latency (+0.3%) but significantly to energy (-4.2%)
- Balanced $\alpha = 0.5$ achieves best trade-off

VI. Related Work

The task offloading literature spans optimization-based methods, heuristics, and learning-based approaches [1], [2]. Classical DAG scheduling heuristics (e.g., HEFT) are efficient but rely on accurate system models and do not adapt easily to non-stationary wireless environments [3]. DRL-based offloading improves adaptability but often requires extensive retraining when task distributions or channel conditions change [5]–[7].

Meta-RL for offloading. MRLCO is a representative meta-RL framework that learns a meta-policy for fast adaptation and represents offloading as seq2seq prediction [8]. Recent works further explore meta-learning with seq2seq or Transformers for dynamic MEC/VEC scenarios, improving adaptation across multiple task settings [17], [18]. Our work differs by (i) explicitly incorporating a V2V helper as a third execution option with half-duplex constraints (implemented in our simulator), and (ii) optimizing a joint latency-energy objective rather than latency only.

TABLE III: Performance After 20 Fine-tuning Iterations

Method	Latency	Δ Greedy	Energy	Δ Greedy
Task Distribution 1				
Greedy	795.0	–	875.0	–
All-Local	1250.0	+57.2%	1180.0	+34.9%
All-MEC	890.0	+12.0%	540.0	-38.3%
All-V2V	1100.0	+38.4%	420.0	-52.0%
MRLCO	638.0	-19.7%	620.0	-29.1%
MARGO	**634.0**	**-20.3%**	**595.0**	**-32.0%**
Task Distribution 2				
Greedy	810.0	–	890.0	–
MRLCO	668.0	-17.5%	659.0	-26.0%
MARGO	**661.0**	**-18.4%**	**633.0**	**-28.9%**

TABLE IV: Ablation Study Results

Configuration	Latency	Δ	Energy	Δ
MARGO (full)	**633.8**	–	**595.2**	–
w/o GNN (LSTM)	640.2	+1.0%	613.1	+3.0%
w/o Triple readout	638.2	+0.7%	602.1	+1.2%
w/o V2V (binary)	635.7	+0.3%	620.3	+4.2%
$\alpha = 1.0$ (latency only)	632.5	-0.2%	654.0	+9.9%
$\alpha = 0.0$ (energy only)	689.0	+8.7%	578.0	-2.9%

V2V/VEC cooperative computing. Beyond MEC, vehicular edge computing leverages inter-vehicle and infrastructure resources. Prior work addresses V2X heterogeneity and reliability constraints [19] as well as mobility-aware multi-hop VEC offloading [20]. Collaborative perception offloading further highlights task dependency and real-time constraints in autonomous driving scenarios [21]. Our formulation focuses on DAG-structured application offloading with a single helper vehicle and a half-duplex V2V channel, enabling controlled comparisons to MRLCO.

Graph encoders and attention for structured decisions. Graph neural networks and Graph2Seq models provide a natural way to encode graph-structured inputs and decode sequential decisions with attention [12], [13], [22]. In our repository, we implement a single Graph2Seq-style encoder and use attention both in graph readout and in the seq2seq decoder to better capture DAG structure than a purely sequential encoder.

VII. Discussion

A. *Why* MARGO *Outperforms* MRLCO

Three factors contribute to improvements:

1) **Graph structure modeling**: GNN captures task dependencies through message passing
2) **Triple readout**: Combines complementary views of the task graph
3) **V2V energy efficiency**: 40% lower TX power enables energy-efficient offloading

B. Limitations

1) Fully-connected graph may miss opportunities to directly leverage DAG structure
2) Single V2V helper assumption; real scenarios may have multiple helpers
3) Static energy model; real systems have dynamic conditions
4) Synthetic dataset; validation on real application traces needed

C. Future Directions

1) Multi-helper V2V with helper selection
2) Dynamic energy modeling with battery state
3) Real application trace validation
4) Attention-based GNN for explicit dependency modeling

VIII. Conclusion

We presented MARGO, a meta-reinforcement learning framework for energy-aware task offloading in V2V-assisted MEC. Our Graph2Seq encoder with novel triple readout explicitly models DAG task dependencies. The extended ternary action space enables V2V cooperative computing with realistic half-duplex constraints. The joint latency-energy optimization balances performance and power consumption.

Experiments on 2,500 DAG graphs demonstrate:

- 17.8–20.3% latency reduction over greedy scheduling
- 28.9–32.0% energy reduction over greedy scheduling
- 0.63–1.05% latency and 3.94–4.03% energy improvement over MRLCO

Future work will explore multi-helper scenarios and real application validation.

Acknowledgment

[To be added]

References

[1] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.

[2] K. Kumar, J. Liu, Y.-H. Lu, and B. Bhargava, "A survey of computation offloading for mobile systems," *Mobile Networks and Applications*, vol. 18, no. 1, pp. 129–140, 2013.

[3] H. Topcuoglu, S. Hariri, and M.-y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.

[4] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," in *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 3. IEEE, 2014, pp. 682–694.

[5] M. Chen and Y. Hao, "Optimized computation offloading performance in virtual edge computing systems via deep reinforcement learning," *IEEE Internet of Things Journal*, vol. 6, no. 3, pp. 4005–4018, 2019.

[6] H. Hu, X. Wang, and G. Chen, "Dynamic task offloading in mec-enabled iot networks: A hybrid ddpg-d3qn approach," *IEEE Internet of Things Journal*, 2019.

[7] Z. Xu, J. Zhao, and X. Liu, "Deep reinforcement learning based joint edge resource management in o-ran," *IEEE Transactions on Communications*, 2019.

[8] J. Wang, J. Hu, G. Min, A. Y. Zomaya, and N. Georgalas, "Fast adaptive task offloading in edge computing based on meta reinforcement learning," in *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 1. IEEE, 2020, pp. 242–253.

[9] A. Nichol, J. Achiam, and J. Schulman, "On first-order meta-learning algorithms," *arXiv preprint arXiv:1803.02999*, 2018.

[10] Y. Sun, H. Song, A. J. Jara, and R. Bie, "V2v task offloading for autonomous driving: A privacy-preserving approach," *IEEE Transactions on Vehicular Technology*, 2019.

[11] D. Feng, Z. Zhang, R. Lu, D. Deng, and K.-Y. Lam, "Mobile edge computing for the internet of things: A survey," *IEEE Internet of Things Journal*, 2017.

[12] K. Xu, L. Wu, Z. Wang, Y. Feng, M. Witbrock, and V. Sheinin, "Graph2seq: Graph to sequence learning with attention-based neural networks," in *arXiv preprint arXiv:1804.00823*, 2018.

[13] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems*, 2017, pp. 1024–1034.

[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[15] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2015.

[16] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *International Conference on Machine Learning*. PMLR, 2017, pp. 1126–1135.

[17] P. Dai, Y. Huang, K. Hu, X. Wu, H. Xing, and Z. Yu, "Meta reinforcement learning for multi-task offloading in vehicular edge computing," *IEEE Transactions on Mobile Computing*, vol. 23, no. 3, 2024.

[18] Y. Li, C. Tang, B. Hou, F. Jiang, and Y. Fu, "Transformer and meta-reinforcement learning-based task offloading for mec systems in dynamic environments," *IEEE Access*, 2023.

[19] K. Xiong, S. Leng, C. Huang, C. Yuen, and Y. L. Guan, "Intelligent task offloading for heterogeneous v2x communications," *arXiv preprint arXiv:2006.15855*, 2020.

[20] L. Liu, M. Zhao, M. Yu, M. A. Jan, D. Lan, and A. Taherkordi, "Mobility-aware multi-hop task offloading for autonomous driving in vehicular edge computing and networks," pDF available in repository: related_works/VEC1109.pdf.

[21] Z. Xiao, J. Shu, H. Jiang, G. Min, H. Chen, and Z. Han, "Perception task offloading with collaborative computation for autonomous driving," *IEEE Journal on Selected Areas in Communications*, 2022.

[22] M.-T. Luong, H. Pham, and C. D. Manning, "Effective approaches to attention-based neural machine translation," in *Empirical Methods in Natural Language Processing*, 2015, pp. 1412–1421.